# Final Project

Ryan Webster

2025-05-05

## Load Data and Libraries

```
credit_risk <- read.csv("credit_customers.csv", stringsAsFactors = T)
library(tidyverse)
library(caret)
library(randomForest)
library(knitr)
library(cluster)
```

## Clean Data

First, lets first check for any NAs in the data

```
any(colSums(is.na(credit_risk)) != 0)
```

```
## [1] FALSE
```

We seem to not have any missing data in the dataset.

Now, we will separate personal_status into two new variables, sex and marital_status as each value in personal_status contains the sex of each subject and their marital status. This was done in order to learn more about the effects of sex and marital status separately rather than the combination of both.

```
credit_risk <- credit_risk %>%
  # Takes the personal_status variable and splits into two new cols, sex and marital status
  separate(personal_status, into = c("sex", "marital_status"), sep = " ")
```

Let's also make sure that our class variable is numeric (1 = Good, 0 = Bad) so that the models won't run into any errors.

```
credit_risk <- credit_risk %>%
  mutate(class = ifelse(class == "good", 1, 0))
```

We should also check for class imbalance in our response variable. This will be important to note when we evaluate model accuracy.

```r
table(credit_risk$class) %>%
  kable(col.names = c("Class", "Count"))
```

| Class | Count |
|---|---|
| 0 | 300 |
| 1 | 700 |

To investigate any potential clusters in the data, we'll use K-Means. However, this means we can only work with continuous data. It's also necessary to scale.

```r
cluster_data <- credit_risk %>%
  select(duration, credit_amount, age, installment_commitment,
         residence_since, existing_credits, num_dependents)
cluster_data_scale <- scale(cluster_data)
```

Applying K-mean

```r
set.seed(1)
kmeans_result <- kmeans(cluster_data_scale, centers = 3)
```

How the 3 different clusters differ:

```r
aggregate(cluster_data, by = list(kmeans_result$cluster), FUN = mean) %>%
  kable(
    col.names = c(
      "Group", "Duration", "Credit Amount", "Age",
      "Installment Commit.", "Residence Since", "Existing Credits",
      "Num Dependents")
  )
```

| Group | Duration | Credit Amount | Age | Installment Commit. | Residence Since | Existing Credits | Num Dependents |
|---|---|---|---|---|---|---|---|
| 1 | 16.24028 | 2418.544 | 46.93286 | 3.063604 | 3.473498 | 1.692579 | 1.385159 |
| 2 | 39.58480 | 8022.971 | 35.27485 | 2.614035 | 2.824561 | 1.444444 | 1.157895 |
| 3 | 17.46886 | 2225.059 | 29.72894 | 3.038461 | 2.525641 | 1.247253 | 1.034798 |

The results will differ every time the document is rendered, but there will be three groups, one with higher duration, credit amount, and middle of the pack age. With background knowledge, they're most likely to have better credit. The other two groups have much lower duration and credit amount, but one is older than the good credit group and one is younger. These two groups would probably have worse credit, but one is considerably older than the other.

## Logistic Regression

The first of three classifiers that will be used for this project is Logistic Regression. We will first start with a model with all variables. In this scenario, we will split up the training data 75/25.

```r
set.seed(1)
# Split the data into 75/25
train_index <- sample(1:nrow(credit_risk), size = 0.75 * nrow(credit_risk))
training_data <- credit_risk[train_index, ]
testing_data <- credit_risk[-train_index, ]
```

Fitting a full model, we see that many of the predictors are insignificant.

```r
logit_reg_full <- glm(class ~ ., data = training_data, family = "binomial")
# summary(logit_reg_full)
# Calculate Full Model's R2
cat(
  "Full Model's R^2:",
  (1 - (logit_reg_full$deviance / logit_reg_full$null.deviance)),
  "\nFull Model's AIC:",
  logit_reg_full$aic, "\n"
)
```

```
## Full Model's R^2: 0.2777238
## Full Model's AIC: 752.3094
```

For the model selection process, the step-wise forward process will be used. This requires a null model in order to give the step-wise function the scope of this process.

```r
# Null model
logit_reg_null <- glm(class ~ 1, data = training_data, family = "binomial")
# Stepwise forward
logit_reg_stepwise <- step(
  logit_reg_null,
  scope = list(lower = logit_reg_null, upper = logit_reg_full),
  direction = "forward"
)
```

```r
# summary(logit_reg_stepwise)
cat(
  "Reduced Model's R^2:",
  (1 - (logit_reg_stepwise$deviance / logit_reg_stepwise$null.deviance)),
  "\nReduced Model's AIC:",
  logit_reg_stepwise$aic,
  "\n"
)
```

```
## Reduced Model's R^2: 0.262535
## Reduced Model's AIC: 738.0689
```

Because of the large number of variables and different categories, the variable summary will not be shown.

With our reduced model, we see that the AIC has decreased by 14 points. The $R^2$ value decreased by 0.01, but this was expected as the $R^2$ value tends to be greater for logistic regression models that have a large number of parameters.

This reduced logistic regression model identified several significant predictors of credit denial, including checking account status, credit history, installment commitment, and credit amount. Notably, applicants

with no checking account, higher installment commitments, or used car purposes had substantially higher odds of denial. However, many variables in the model (e.g., purpose categories, some employment levels) were not statistically significant.

In making predictions, we will classify probability greater than 50% as good credit and less as bad credit.

```
# Make predictions on testing data, classify by 0.5, create confusion matrix
logit_pred <- predict(
  logit_reg_stepwise,
  newdata = testing_data,
  type = "response"
)
logit_pred_class <- ifelse(logit_pred > 0.5, 1, 0)
confusionMatrix(
  factor(logit_pred_class),
  factor(testing_data$class),
  positive = "1"
)
```

Table 3: Confusion Matrix for Threshold = 0.5

| | **Actual** | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 41 | 25 |
| 1 | 40 | 144 |

Accuracy = 0.736, Sensitivity = 0.8521, Specificity = 0.4938

The initial accuracy using 0.5 as a threshold for classification is 73.6%. However, the specificity is low at 0.4938. Meaning around 50% of subjects who were labeled as at-risk, weren't at risk. Because both false positives and negatives can be very consequential in this scenario, let's determine the best threshold that maximizes the Youden index (Sensitivity + Specificity - 1)

```
maximize_youden <- function(predictions) {
  thresholds <- seq(0, 1, by = 0.01)

# Initialize data frame to store results
results <- data.frame(
  threshold = thresholds,
  sensitivity = rep(0, length(thresholds)),
  specificity = rep(0, length(thresholds)),
  youden_index = rep(0, length(thresholds))
)

# Actual class labels
actual <- factor(testing_data$class)

for (i in seq_along(thresholds)) {
  # Find threshold, predict by threshold and extract sensitivity and specificity
  # data from the resulting confusion matrix. Calculate youden index.
  t <- thresholds[i]
  predicted_class <- factor(ifelse(predictions > t, 1, 0))
  cm <- confusionMatrix(predicted_class, actual, positive = "1")
```

```
  results$sensitivity[i] <- cm$byClass["Sensitivity"]
  results$specificity[i] <- cm$byClass["Specificity"]
  results$youden_index[i] <- results$sensitivity[i] + results$specificity[i] - 1
}

# Extract best threshold based on max youden index
best_row <- results[which.max(results$youden_index), ]
return(best_row)
}

best_row <- maximize_youden(logit_pred)
best_row
```

```
##    threshold sensitivity specificity youden_index
## 64      0.63   0.7928994   0.6419753    0.4348747
```

Re-prediciting with the new threshold, 0.63:

```
best_thresh <- best_row$threshold
logit_pred_class <- ifelse(logit_pred > best_thresh, 1, 0)
confusionMatrix(
  factor(logit_pred_class),
  factor(testing_data$class),
  positive = "1"
)
```

Table 4: Confusion Matrix for Threshold = 0.63

| | **Actual** | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 52 | 35 |
| 1 | 29 | 134 |

Accuracy: 0.744, Sensitivity: 0.7929, Specificity: 0.642

We see that accuracy increases, along with a major improvement in the specificity by sacrificing sensitivity. We'll also calculate mean square error to validate between models.

```
mean((logit_pred_class - testing_data$class) ^ 2)
```

```
## [1] 0.256
```

## Random Forest

Next, we'll use a Random Forest model. The model will be trained using 500 trees.

```
set.seed(1)
random_forest <- randomForest(
  as.factor(class) ~ .,
  data = training_data,
```

```
  mtry = dim(training_data)[2] - 1,
  importance = TRUE,
  ntree = 500
)
```

Next, we'll first classify using a 0.5 threshold

```
y_hat <- predict(random_forest, newdata = testing_data, type = "prob")
random_forest_class <- ifelse(y_hat[, 2] > 0.5, 1, 0)
confusionMatrix(
  as.factor(random_forest_class),
  as.factor(testing_data$class),
  positive = "1"
)
```

| | **Actual** | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 39 | 20 |
| 1 | 42 | 149 |

Accuracy: 0.752, Sensitivity: 0.8817, Specificity: 0.4815.

Again, we'll tune the threshold due to the class imbalance and low sensitivity.

```
best_row <- maximize_youden(y_hat[, 2])
best_row
```

```
##    threshold sensitivity specificity youden_index
## 76      0.75   0.5502959   0.8518519    0.4021477
```

Re-predict with new threshold, 0.75:

```
best_thresh <- best_row$threshold

random_forest_class <- ifelse(y_hat[, 2] > best_thresh, 1, 0)
confusionMatrix(
  as.factor(random_forest_class),
  as.factor(testing_data$class),
  positive = "1"
)
```

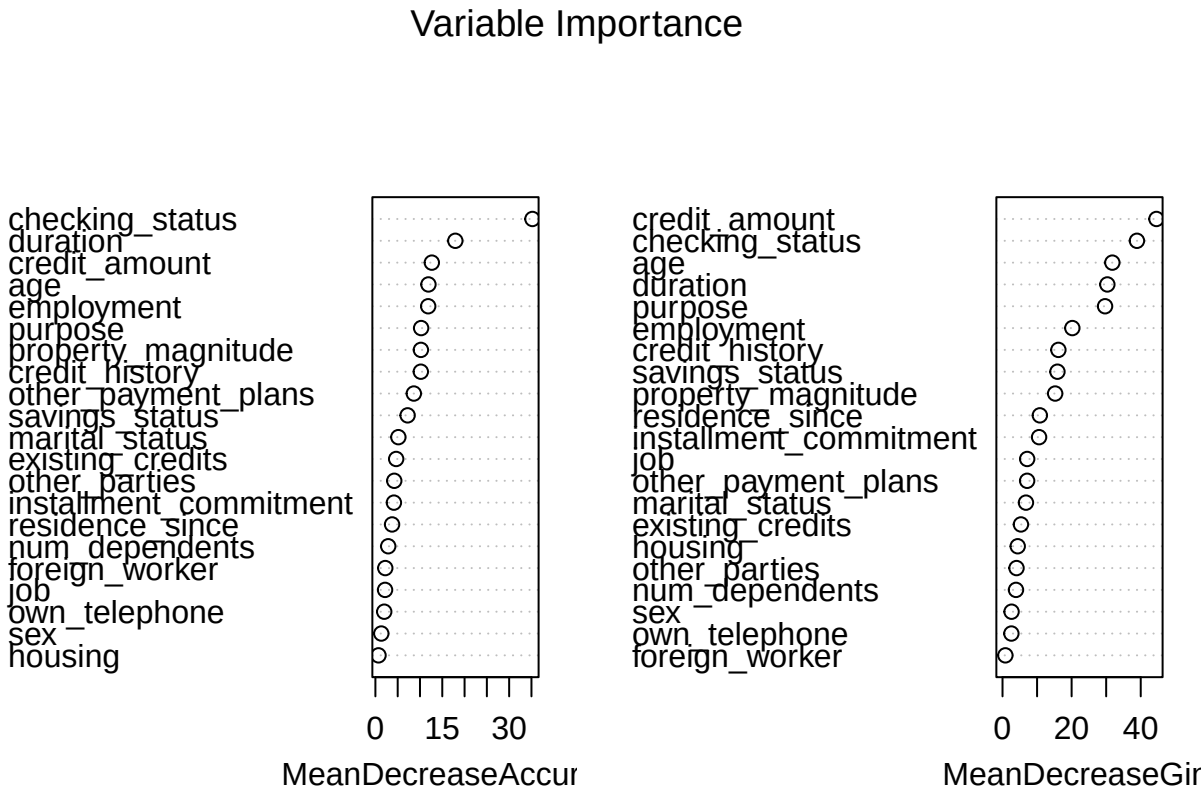
| | **Actual** | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 69 | 76 |
| 1 | 12 | 93 |

Accuracy: 0.648, Sensitivity: 0.5503, Specificity: 0.8519

By classifying by a new threshold, we are sacrificing some accuracy and sensitivity to boost specificity.

With Random Forest, one can obtain a variable importance plot, which will show how much the model's accuracy and Gini will decrease if removed.

```r
varImpPlot(random_forest, main = "Variable Importance")
```




We find that checking status, duration, credit amount, age, purpose, and employment are all relatively important in this Random Forest model.

```r
mean((as.numeric(random_forest_class) - testing_data$class)^2)
```

```
## [1] 0.352
```

## KNN

Next, we'll fit a K Nearest Neighbors (KNN) onto the training data with the variables we deemed important in the Random Forest section. With KNN, it's important to first scale all numeric data first.

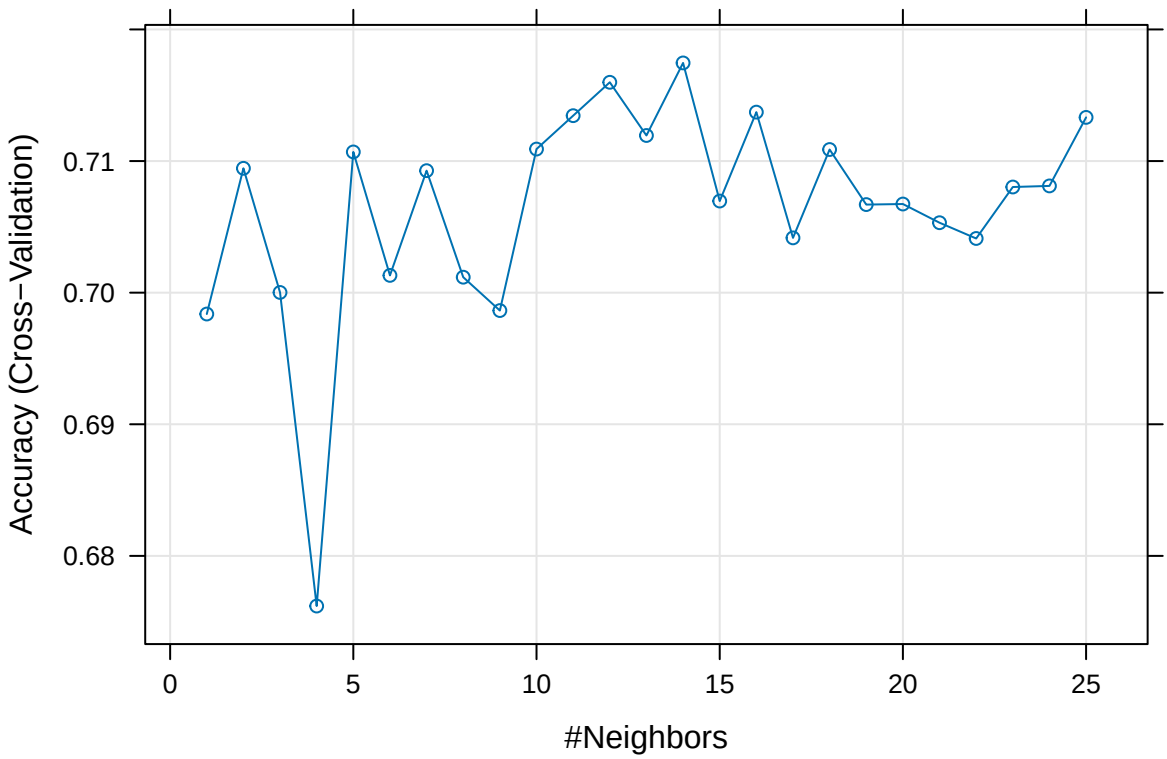
```r
# Store variables to scale
scale_vars <- c("duration", "credit_amount", "installment_commitment", "residence_since", "age", "existi

# Scale
credit_risk_scaled <- credit_risk %>%
  mutate(across(all_of(scale_vars), scale))

training_data <- credit_risk_scaled[train_index, ]
testing_data <- credit_risk_scaled[-train_index, ]
```

When fitting the KNN, we'll use cross validation to find the optimal K from 1 to 25.

```
set.seed(1)
fitControl <- trainControl(method = "cv",number = 25)
knn <- train(
  as.factor(class) ~ credit_amount + checking_status + age + duration + purpose + employment,
  method = "knn",
  tuneGrid = expand.grid(k = 1:25),
  trControl = fitControl,
  metric = "Accuracy",
  data = training_data
)
```

Plotting the different cross validation accuracies by K:

```
plot(knn)
```




We find that the accuracy is maximized when:

```
cat("Best K:", knn$bestTune[1, 1])
```

```
## Best K: 14
```

```r
knn_pred_prob <- predict(knn, newdata = testing_data, type = "prob")
knn_pred_class <- ifelse(knn_pred_prob[, 2] > 0.5, 1, 0)
confusionMatrix(
  as.factor(knn_pred_class),
  as.factor(testing_data$class),
  positive = "1"
)
```

| | Actual | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 28 | 23 |
| 1 | 53 | 146 |

Accuracy: 0.696, Sensitivity: 0.8639, Specificity: 0.3457

Like with the other models, we'll find an optimal threshold

```r
# Extract best threshold based on max youden index
best_row <- maximize_youden(knn_pred_prob[, 2])
best_row
```

```
##    threshold sensitivity specificity youden_index
## 73      0.72    0.556213   0.8395062    0.3957192
```

Refit with new threshold, 0.72:

```r
best_thresh <- best_row$threshold
knn_pred_prob <- predict(knn, newdata = testing_data, type = "prob")
knn_pred_class <- ifelse(knn_pred_prob[, 2] > best_thresh, 1, 0)
confusionMatrix(
  as.factor(knn_pred_class),
  as.factor(testing_data$class),
  positive = "1"
)
```

| | Actual | |
|---|---|---|
| **Predictions** | 0 | 1 |
| 0 | 68 | 75 |
| 1 | 13 | 94 |

Accuracy: 0.648, Sensitivity: 0.5562, Specificity: 0.8395.

By readjusting the threshold, we sacrifice 5% of model accuracy and sensitivity in order to greatly improve specificity and creating a much more equal trade-off of false positives and negatives.

The accuracy was equivalent to the Random Forest's. So its MSE will be equivalent:

```r
# MSE
mean((as.numeric(knn_pred_class) - testing_data$class)^2)
```

```
## [1] 0.352
```

## Comparison

Shown below is a comparison of the three different models using the maximized Youdin threshold.

| Model | Accuracy | MSE | Sensitivity | Specificity |
|---|---|---|---|---|
| Logistic Regression | 74.4% | 0.256 | 0.7929 | 0.642 |
| Random Forest | 64.8% | 0.352 | 0.5503 | 0.8519 |
| KNN | 64.8% | 0.352 | 0.5562 | 0.8395 |

Logistic Regression is the best overall performer among the three models. It achieves the highest accuracy at 74.4%, the lowest mean squared error (MSE) at 0.256, and the highest sensitivity at 0.7929. While its specificity of 0.642 is lower than that of the other models, this trade-off is acceptable if you're more focused on reducing false-negatives over false-positives. In comparison, both Random Forest and KNN have lower accuracy (64.8%) and higher MSE (0.352). Although they exhibit higher specificity (around 0.84–0.85), their sensitivity scores are considerably lower, making them less effective at identifying true positives. Therefore, Logistic Regression provides the best overall balance of performance metrics in classifying credit risk.